

Research on stream processing engine and benchmarking framework

Qionghua Le

Mingang Chen

Wenjie Chen

Shanghai Key Laboratory of Computer
Software Testing & Evaluating

Shanghai Key Laboratory of Computer
Software Testing & Evaluating

Shanghai Key Laboratory of Computer
Software Testing & Evaluating

Shanghai Development Center of
Computer Software Technology

Shanghai Development Center of
Computer Software Technology

Shanghai Development Center of
Computer Software Technology

Shanghai, China

Shanghai, China

Shanghai, China

lqh@sscenter.sh.cn

cmg@sscenter.sh.cn

cwj@sscenter.sh.cn

Abstract—Stream computing engine is an important part of big data system, and benchmarking is one of the main means to measure the engine's performance. In this paper, we compare the differences between two engines, Spark Streaming and Flink, in stream processing technologies. Then the open source benchmarking frameworks supporting stream processing and their respective characteristics are studied, and the HiBench testing framework is selected to test the two stream processing engines. The test results show that Flink is better than Spark Streaming in terms of performance in shuffle, stateful computation and windowed computation.

Keywords—stream processing engine ,Spark Streaming, Flink ,benchmarking framework, HiBench

I. INTRODUCTION

Data has high business value at the initial stage of generation, but the value of data is not constant, it will rapidly decrease with the passage of time. The traditional batch processing model processes data in a specific time cycle, and the timeliness can no longer meet the business requirements. stream processing is a processing model that does computation on continuous data, and its advantage is that it can uncover the value that exists in the data in a shorter time.

The development of stream processing engines has expanded the ecological architecture of big data systems, but also poses challenges to the testing of big data systems. First, there is no industry standard test benchmark in the field of stream computing, and some benchmarks that are widely used for testing big data systems are not applicable to stream computing engines, such as TPC-C [1], TPC-H [2], Hadoop-related MRBench [3], Gridmix [4], etc. Second, testing of stream computing engines needs to be tailored to the application scenarios of stream computing. Third, due to the different focus, it is necessary to choose suitable test metrics to correctly and objectively reflect the characteristics of stream computing engines. How to reasonably evaluate stream computing engines has become a new direction for big data system testing research.

Benchmarking is a typical activity to measure and evaluate software performance metrics, and it is an important

tool to evaluate the performance of big data systems, which can provide a basis for system performance evaluation and identify system risks and performance bottlenecks as early as possible.

This paper compares the differences between Spark Streaming and Flink in stream processing processing, using the HiBench testing framework as a testing tool for both engines, comparing their performance in shuffle, stateful computation and windowed computation.

II. STREAM PROCESSING ENGINE

After the concept of stream processing emerged, many large technology companies, such as IBM, Twitter, Apache, etc., have invested in stream processing engine development projects. The two most popular and widely used stream processing engines are Spark Streaming and Flink, both of which are open source projects of Apache, and both Spark and Flink are the most active projects of Apache community. In this paper, we will use these two engines as the object of stream benchmarking, and this section introduces the main differences between the two engines in stream processing.

Spark is a general distributed memory computing framework, and Spark Streaming is an extension of Spark core API to achieve scalable, high-throughput and highly reliable real-time data stream processing based on SparkCore. SprakStreaming splits the received real-time stream data into multiple data blocks at certain intervals, called "micro-batch", to simulate streams in the form of micro-batches, and implements stream processing with a series of short, stateless, deterministic batches. It is essentially batch processing.

Flink's basic data model is data streams, and it is a true stream processing engine with native support for unbounded data processing. The "state" can be understood as the intermediate result of data processing, and if the result of processing an event is related to the previously processed data, it is called stateful computation. Flink is designed to support stateful computation and provides a unified state management scheme, storing the state inside the Flink engine, reducing the engine's dependency on external components and improving performance. The differences between Spark Streaming and Flink in stream processing are summarized in TABLE I.

TABLE I. DIFFERENCES BETWEEN SPARK STREAMING AND FLINK

	Spark Streaming	Flink
Streaming mode	micro bench	Native
Stateful	Yes (Dstream)	Yes (Operator)
Latency	second	millisecond
Fault Tolerance	Checkpoint based on RDD	Checkpoint + Savepoint
Throughput	High	High
Guarantees	Exactly-Once	Exactly-Once
Delay & out-of-order	Watermark	Watermark+AllowedLateness+SideOutput
Window	Time-based Sliding window	Time/Count-based Tumbling Window 、 Sliding Window 、 Session Window
Time	ProcessingTime	EventTime、 ProcessingTime

III. STREAM PROCESSING BENCHMARKING FRAMEWORK

We examined the current research findings in engine performance comparison, such as Yassir SAMAD et al. compare Hadoop and Spark performance based on criteria such as execution time, throughput, and acceleration [5]; MZ Dai et al. compare and evaluate the differences in the working patterns of Hadoop, Spark, and Flink on the same big data load [6]. Their studies do address stream computing loads but not specific to stream computing scenarios. Sakr S et al. selected real-world use cases in different domains to compare the performance of Flink, Storm, Heron, and Kafka engines [7]; Jeyhun Karimov et al. chose an online video game scenario and did tests on stateful computation with Storm, Spark Streaming, and Flink, testing the latency of the three engines in different configuration states [8]. Since there is currently no industry standard test benchmark in the field of stream computing, in most of these comparative testing studies of stream computing engines, the authors have chosen application-specific examples and data, and their findings provide a reference for the performance of the engines, but are not easily reproducible. We hope to provide developers and researchers of stream computing applications with a measurement methodology that they can apply to their own systems.

Benchmarking is a type of performance testing that differs from general performance testing in that it has a shorter regression period and intuitive trend analysis. The main role of benchmark testing is to establish a reference benchmark for system performance, provide a reference basis for system parameter configuration and performance optimization, check whether the system achieves the expected performance, establish a long-term source of performance data statistics and compare benchmarks. This section introduces the more widely used open source benchmark testing frameworks that support stream processing and the features of each framework.

Yahoo Streaming Benchmarks (YSB) [9] is an open source stream benchmarking framework developed by the Yahoo Storm team, supporting Storm, Spark, Flink, and Apex engine testing, with a framework simulating a simple advertising scenario. The disadvantage is that Yahoo's

benchmarking design is too simple to test the performance of the engine's state management and Checkpoint.

BigDataBench [10] is an open source collection of big data benchmark tests across 3 domains of system, architecture, and data management developed by Chinese Academy of Sciences. In BigDataBench5.0 Streaming test suite is added which support Spark Streaming and Storm testing. Three types of test load are designed for each engine, but the test scenarios of the load is not corresponding, and the performance comparison test between engines cannot be realized. The output metrics of the test loads are also different, so it is hard to compare the performance differences across test loads.

StreamBench [11] is a benchmarking framework for stream processing designed by Aalto University School of Science. StreamBench is designed with three workloads: WordCount evaluates the performance of the stream processing engine in executing basic operators; AdvClick is used to test the Join operation of two data streams; and K-Means uses K-Means algorithm to test the performance of iteration operator. The three loads are implemented accordingly on different engines, and different engines can be compared in tests that complete the same function.

NEXMark is a testing framework for stream processing engines designed and implemented by Apache Beam based on the Oregon Health & Science University thesis [12] and then expanded and improved by Alibaba [13]. The test framework simulates an online auction system containing three data streams, a static data source and a total of 16 test loads using ANSI SQL standard syntax. The number of loads is rich, but NEXMark currently only supports Flink.

HiBench is a comprehensive big data testing benchmark proposed by Intel's big data group [14]. The whole set of test benchmarks includes micro benchmarks, machine learning, SQL, web search benchmark, graph benchmark and streaming benchmark, involving 24 benchmark loads in 6 directions. HiBench added a test suite for stream processing after version 5.0, and the source code of the test framework is open source on GitHub with over 13,000 stars. The version 7.1.1 was released in July 2020 and the master branch has been kept up to date.

The fourth and fifth chapter will introduce the HiBench Streaming Benchmark framework running process and use the test framework for Spark Streaming and Flink testing and analysis.

IV. HIBENCH

HiBench Streaming Benchmark support for testing Spark Streaming, Flink and Storm, the test environment supports Hadoop2.x, Hadoop3.0, CDH5, CDH6, HDP. HiBench test load execution process is shown in Fig.1 (with Flink as an example)

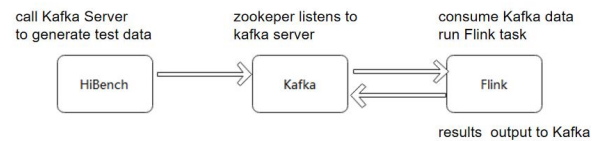


Fig. 1. HiBench test execute process

First HiBench call Kafka Server to generate test data through seed generation script(genSeedDataset) and data generation script(dataGen)(can increase the number of data generated by running multiple processes).Then Zookeeper is responsible for managing the kafka broker , the brokers automatically achieve load balancing and control the partition of each topic.Flink associates the Kafka server through the setting of the HiBench, the data is consumed by Flink to run the corresponding test load, and then generate new data output to Kafka.The test suite can obtain the performance indicators of the test process by query the input and output results.HiBench's benchmark tests for stream processing have four loads as shown in TABLE II.

TABLE II. HIBENCH STREAMING BENCHMARKS

Test Loads	Description
Identity	This workload reads input data from Kafka and then writes result to Kafka immediately, there is no complex business logic involved.
Repartition	This workload reads input data from Kafka and changes the level of parallelism by creating more or fewer partitions. It tests the efficiency of data shuffle in the streaming frameworks.
Stateful Wordcount	This workload counts words cumulatively received from Kafka every few seconds. This tests the stateful operator performance and Checkpoint cost in the streaming frameworks.
Fixwindow	The workloads performs a window based aggregation. It tests the performance of window operation in the streaming frameworks.

V. HIBENCH TEST PROCESS AND RESULTS

Experiments were tested on Spark Streaming and Flink using HiBench 7.1.1 master test suite StreamingBench [15], the test environment is shown in TABLE III~V.

TABLE III. SERVERS CONFIGURATION

Host	ip	OS	cpu
Cloudera-node1(Master)	192.168.71.154	CentOS 6.7	1cpu 4cores 2194MHz
Cloudera-node2	192.168.71.157	CentOS 6.7	2cpu 8cores 2294MHz
Cloudera-node3	192.168.71.173	CentOS 6.7	2cpu 8cores 2294MHz
Cloudera-node4	192.168.71.181	CentOS 6.7	2cpu 8cores 2294MHz

TABLE IV. BIG DATA SYSTEM COMPONENT CONFIGURATION

HDFS 2.6.0 (CDH5.10)	1NameNode + 3DataNode
Zookeeper 3.4.5 (CDH5.10)	3Server
Kafka 0.11.0.3	3broker 24partition

TABLE V. ENGINE UNDER TEST CLUSTER CONFIGURATION

Spark 2.4.0	1Master + 3Worker Memory per Executor 16g 24parallelism
Flink 1.7.1	1JobManager + 3TaskManager Memory per TaskManager 16g 24parallelism

Before starting the test, the engine cluster under test is running normally and all computing resources are free, and run the test suite tool to generate seeds to HDFS.The test starts with Kafka generating test data according to the seed, 200 bytes per data. The Kafka topic data is cleared before each test, and each load is executed for 5 minutes, repeated several times, and the results are averaged.

HiBench load test output contains 1 throughput metric and 10 latency metrics. Latency metrics include average latency,maximum latency, minimum latency, latency standard deviation, and 50% ~99.9% request latency.From these, we selected throughput, average latency and 99% request latency as comparison indexes.The test results are shown in Fig 2 and Fig 3:

The Identity test load does not involve business logic, the computing engine receives kafka data and outputs directly to Kafka, the test results can be used as a reference for the maximum performance of the engine.

The Repartition test load tests the engine shuffle performance by changing the number of partitions.Flink uses rebalance operator to implement repartitioning, and random polling is used to send data during repartitioning;Spark Streaming uses the reparation operator, and the repartitioning process uses hash function.Spark Streaming throughput drops significantly after reaching 60,000 data per second in Repartition tests, processing less than half of the total amount of data and significantly increasing latency to the minute level. Flink throughput is basically comparable to the total amount of data, with an average latency of 78 ms at 100,000 data per second and 99% of requests with latency below 1.5 seconds.

The Wordcount test load tests the performance of the stateful computation of the engine. Flink comes with unified state management, and the corresponding operator is called directly during the implementation, while Spark Streaming requires additional definition of the state processing. From the test results, Flink maintains a stable throughput and average millisecond latency, while Spark Streaming's data processing capability has significantly decreased. This shows that Flink outperforms Spark Streaming in the processing of stateful computations.

The Fixwindow test load tests the performance of window operation, Spark Streaming and Flink use time-based sliding window in the test load, the window length and step length are 10000ms. Flink uses processing time based window in this load, so no additional window calculation will be triggered by Watermark mechanism. The number of window calculations triggered by the two engines should be comparable. The performance of Spark Streaming in the window test is better than that of repartitioning and stateful computation, and the difference in latency with Flink is smaller, probably benefit from Spark's batch processing capability.

From the overall analysis of the test results, Flink outperforms Spark Streaming in the overall capability of stream processing. In this test environment, Flink's data throughput is basically the same as the generated data, and slightly decreases after the data volume reaches 100,000 times per second, and the Repartition test, stateful computation test and fixwindow test are almost the same as the Identity test in terms of throughput change trend. On the contrary, Spark Streaming shows significant throughput

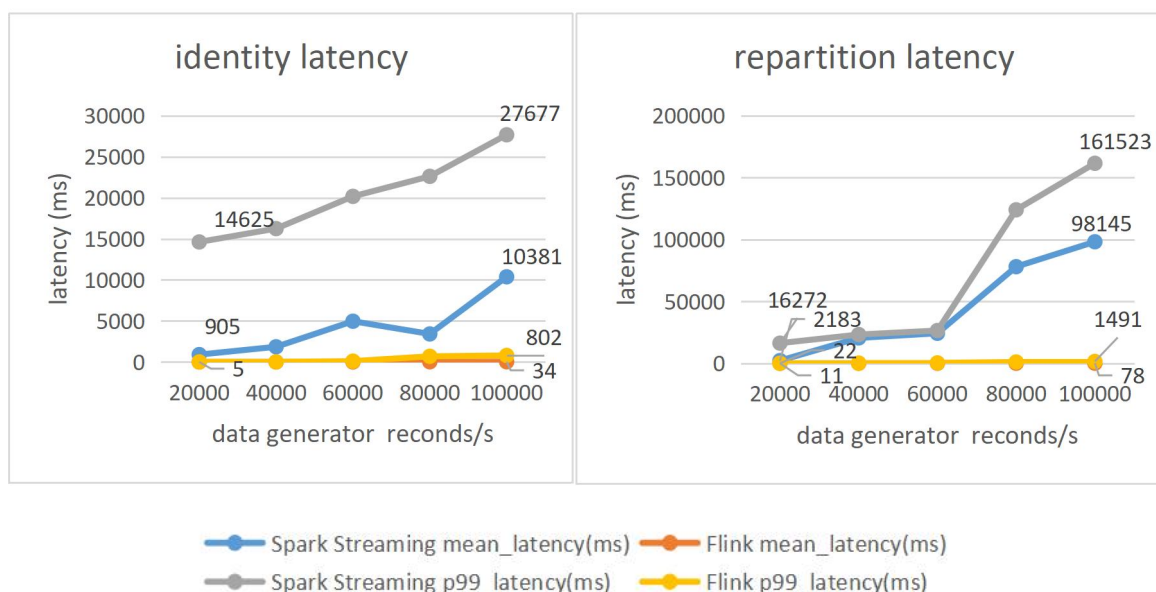
drops in different loads. In terms of latency metrics, Flink's advantage is more obvious, as the Repartition test and stateful computation data are very close to the Identity results, while in the window test latency increases, with the average latency within 10 seconds.

In addition, Spark Streaming's back pressure mechanism may also be a factor in the significant performance degradation after the data volume increases. The backpressure mechanism of Spark Streaming is relatively

simple, mainly calculating a maximum offset based on the execution of the task at the backend, and controlling the speed of Spark Streaming in pulling data from Kafka. Flink is more detailed in handling backpressure, the backpressure is determined by the ratio of sampling and blocking for a certain period of time, and the control of data flow rate is fed from the operator where backpressure occurs up to the data source step by step, so Flink has a higher utilization of resources when facing backpressure, and the engine has a better resistance to pressure.



Fig. 2. HiBench Throughput test results



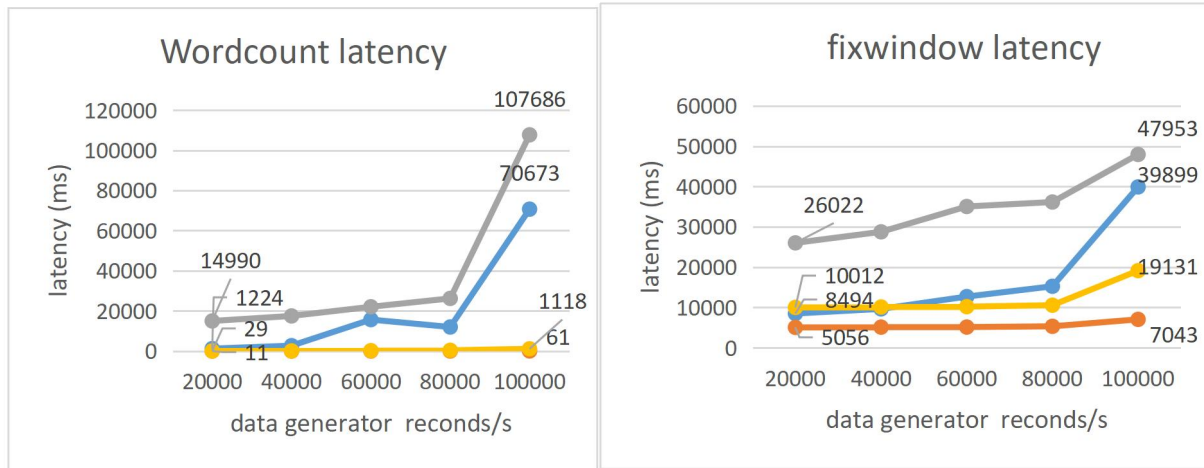


Fig. 3. HiBench Delay test results

VI. SUMMARY

In this paper, we analyze the differences between Spark Streaming and Flink engines in stream processing processing, and study the more widely used open source benchmarking frameworks on the market that support stream processing. Spark Streaming and Flink are tested separately using the HiBench Streaming Benchmark. The test results show that Flink is better than Spark Streaming in terms of performance in shuffle, stateful computation and windowed computation, especially in latency metrics, indicating that Flink is suitable for application in big data systems that are more sensitive to the timeliness of data processing.

Throughout the benchmarking framework for stream processing, including HiBench, there are still areas that need improvement : Test load is not comprehensive enough, insufficient coverage of stream processing scenarios (case of multiple data streams, case of stream data associated with static data, case of window association, etc.); The version of the stream processing engine is updated quickly, and most testing frameworks do not support new versions of the engine; Add evaluation metrics (e.g., reflecting resource utilization, calculation accuracy, etc.) to help reflect engine performance comprehensively.

ACKNOWLEDGMENT

The authors would like to thank the reviewers for their suggestions.

REFERENCES

- [1] TPC-C.[EB/OL]. <http://www.tpc.org/tpcc/>.
- [2] TPC-H.[EB/OL]. <http://www.tpc.org/tpch/>.
- [3] Kim K , Jeon K , Han H , et al. MRBench: A Benchmark for MapReduce Framework[C]// International Conference on Parallel & Distributed Systems. DBLP, 2008.
- [4] Gridmix.[EB/OL].<https://hadoop.apache.org/docs/r1.2.1/gridmix.html>.
- [5] Y. Samadi, M. Zbakh and C. Tadonki, Comparative study between Hadoop and Spark based on Hibench benchmarks [C]//2016 2nd International Conference on Cloud Computing Technologies and Applications (CloudTech), Marrakech : IEEE,2016
- [6] Dai M Z , Gao S F . Framework Performance Evaluation Based on Hadoop,Spark and Flink Large-scale Data Analysis[J]. Journal of China Academy of Electronics and Information Technology, 2018.
- [7] Sakr S S . Comparative Evaluation for the Performance of Big Stream Processing Systems.[EB/OL].<https://xueshu.baidu.com/usercenter/paper/show?paperid=1a6ce2280aca09d3e115be1d3706268a&site=xueshu>
- [8] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen and V. Markl, Benchmarking Distributed Stream Data Processing Systems, [C]//2018 IEEE 34th International Conference on Data Engineering (ICDE), Paris:IEEE,2018
- [9] Yahoo Storm Team. Yahoo streaming benchmark. [EB/OL]. <http://yahoostorm.tumblr.com/post/135321837876/benchmarking-streaming-computation-engines-at>.
- [10] Lei Wang; Jianfeng Zhan; Chunjie Luo,et al. BigDataBench: A big data benchmark suite from internet services [C]// 2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA).Orlando:IEEE,2014
- [11] Wang Y . Stream Processing Systems Benchmark: StreamBench[EB/OL].<https://developer.aliyun.com/article/772485?spm=a2c6h.12873581.0.dArticle772485.83512634JhCnJ>
- [12] Tucker P , Tufte K , Papadimos V , et al. NEXMark - A Benchmark for Queries over Data Streams DRAFT. 2008.
- [13] Nexmark:.[EB/OL]<https://github.com/nexmark/nexmark>.
- [14] Huang S , Jie H , Dai J , et al. The HiBench benchmark suite: Characterization of the MapReduce-based data analysis[C]// 2010 IEEE 26th International Conference on Data Engineering Workshops (ICDEW 2010). IEEE, 2010.
- [15] Intel-bigdata/HiBench.[EB/OL].<https://github.com/Intel-bigdata/HiBench/tree/master>.